

ELECTRONICS ENGINNERING

MICROPROCCESSORS LABORATORY

2023-2024 FALL

EXPERIMENT-2

Group Members

Name	Fouad	Büşra	Ayşe Beril
Surname	Aladhami	Ülker	Çevik
Student ID	210102002808	200102002011	2101020002020

1 INTRODUCTION 3

1.1 RCC Configuration I/O port clock enable register (RCC_IOPENR) 3
1.2 Peripherals addresses 3
1.3 GPIO MODER Register 4
1.4 GPIO ODR register 4
1.5 GPIO IDR Register 5

2 PROBLEM 1 5

2.1 Flowchart 6
2.2 Code 6

3 PROBLEM 2 9

3.1 Schematic 9
3.2 Flowchart 10
3.3 Code 10

4 PROBLEM 3 13

4.1 Schematic 13
4.2 Flowchart 14
4.3 Code 14

5 PROBLEM 4 17

5.1 Schematic 18
5.2 Flowchart 18
5.3 Code 20

6 PROBLEM 5 24

6.1 Schematic 25
6.2 Flowchart 26
6.3 Code 27

1 Introduction

1.1 RCC Configuration I/O port clock enable register (RCC_IOPENR)

The register shown in Figure 1 is defined as RCC_IOPENR, reset and clock setting is activated by making the related bits 1.

5.4.13 I/O port clock enable register (RCC_IOPENR)

Address: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOFEN	GPIOEEN(1)	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
										rw	rw	rw	rw	rw	rw

1. Only significant on devices integrating the corresponding peripheral, otherwise reserved. Refer to *Section 1.4: Availability of peripherals*.

Bits 31:6 Reserved, must be kept at reset value.

Bit 5 **GPIOFEN:** I/O port F clock enable
This bit is set and cleared by software.
0: Disable
1: Enable

Bit 4 **GPIOEEN:** I/O port E clock enable(1)
This bit is set and cleared by software.
0: Disable
1: Enable

Bit 3 **GPIODEN:** I/O port D clock enable
This bit is set and cleared by software.
0: Disable
1: Enable

Figure 1: RCC_IOPENR register

1.2 Peripherals addresses

GPIOX base addresses were defined as GPIOX_BASE and these addresses were accessed to the registers that make the settings on the X ports. It is seen that there is an offset of 0x400 between the ports.

Table 6. STM32G0x1 peripheral register boundary addresses

Bus	Boundary address	Size	Peripheral	Peripheral register map
-	0xE000 0000 - 0xE00F FFFF	1MB	Cortex®-M0+ internal peripherals	-
IOPORT	0x5000 1800 - 0x5FFF FFFF	~256 MB	Reserved	-
	0x5000 1400 - 0x5000 17FF	1 KB	GPIOF	*Section 7.4.12 on page 248*
	0x5000 1000 - 0x5000 13FF	1 KB	GPIOE	*Section 7.4.12 on page 248*
	0x5000 0C00 - 0x5000 0FFF	1 KB	GPIOD	*Section 7.4.12 on page 248*
	0x5000 0800 - 0x5000 0BFF	1 KB	GPIOC	*Section 7.4.12 on page 248*
	0x5000 0400 - 0x5000 07FF	1 KB	GPIOB	*Section 7.4.12 on page 248*
	0x5000 0000 - 0x5000 03FF	1 KB	GPIOA	*Section 7.4.12 on page 248*

Figure 2: GPIOX address map

1.3 GPIO MODER Register

Using GPIOX_BASE addresses, the modes of the pins in the MODER registry can be set by accessing the MODER registry with offset 0x00.

7.4.1 GPIO port mode register (GPIOx_MODER) (x =A to F)

Address offset:0x00

Reset value: 0xEBFF FFFF for port A

Reset value: 0xFFFF FFFF for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODE[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

Figure 3: GPIOx_MODER Register

1.4 GPIO ODR register

Using GPIOX_BASE addresses, the ODR register is accessed with 0x14 Offsett and the pins are powered by assigning a value of 1 to the relevant bits in the ODR register.

7.4.1 GPIO port mode register (GPIOx_MODER) (x =A to F)

Address offset:0x00

Reset value: 0xEBFF FFFF for port A

Reset value: 0xFFFF FFFF for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODE[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

Figure 4: GPIOx_ODR register

1.5 GPIO IDR Register

Using GPIOX_BASE addresses, the IDR register is accessed with 0x10 Offset and the pins state can read by read this register.

7.4.5 GPIO port input data register (GPIOx_IDR) (x = A to F)

Address offset: 0x10

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

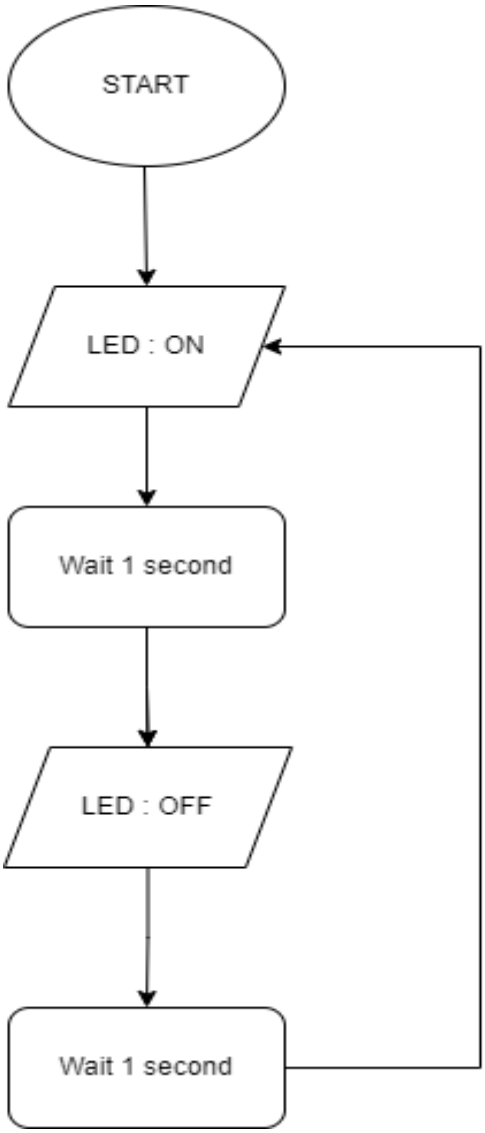
Bits 15:0 **ID[15:0]:** Port x input data I/O pin y (y = 15 to 0)
These bits are read-only. They contain the input value of the corresponding I/O port.

Figure 5: GPIOx_IDR register

2 Problem 1

The onboard led is connected to PC6 pin. The led is blinking by set the pin high and low with 1s intervals.

2.1 Flowchart



2.2 Code

```
/*
* asm.s
*
* author: Fouad Aladhami/Büşra Ülker/Ayşe Beril Cevik
*
* description: Added the necessary stuff for turning on the green LED on the
*   G031K8 Nucleo board. Mostly for teaching.
*/
.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb
/* make linker see this */
```

```
.global Reset_Handler
/* get these from linker script */
.word _sdata
.word _edata
.word _sbss
.word _ebss
/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
.equ RCC_BASE,        (0x40021000)        // RCC base address
.equ RCC_IOPENR,      (RCC_BASE   + (0x34)) // RCC IOPENR register offset
.equ GPIOC_BASE,      (0x50000800)        // GPIOC base address
.equ GPIOC_MODER,     (GPIOC_BASE + (0x00)) // GPIOC MODER register offset
.equ GPIOC_ODR,       (GPIOC_BASE + (0x14)) // GPIOC ODR register offset
.equ DELAY_FREQ,                (16000000/3)
/* vector table, +1 thumb mode */
.section .vectors
vector_table:
        .word _estack           /*     Stack pointer */
        .word Reset_Handler +1    /*     Reset handler */
        .word Default_Handler +1  /*      NMI handler */
        .word Default_Handler +1  /* HardFault handler */
        /* add rest of them here if needed */
/* reset handler */
.section .text
Reset_Handler:
        /* set stack pointer */
        ldr r0, =_estack
        mov sp, r0
        /* initialize data and bss
         * not necessary for rom only code
         * */
        bl init_data
        /* call main */
        bl main
        /* trap if returned */
        b .
/* initialize data and bss sections */
.section .text
init_data:
        /* copy rom to ram */
        ldr r0, =_sdata
        ldr r1, =_edata
        ldr r2, =_sidata
        movs r3, #0
        b LoopCopyDataInit
        CopyDataInit:
                ldr r4, [r2, r3]
```

```
                str r4, [r0, r3]
                adds r3, r3, #4
        LoopCopyDataInit:
                adds r4, r0, r3
                cmp r4, r1
                bcc CopyDataInit
        /* zero bss */
        ldr r2, =_sbss
        ldr r4, =_ebss
        movs r3, #0
        b LoopFillZerobss
        FillZerobss:
                str  r3, [r2]
                adds r2, r2, #4
        LoopFillZerobss:
                cmp r2, r4
                bcc FillZerobss
        bx lr
/* default handler */
.section .text
Default_Handler:
        b Default_Handler
/* main function */
.section .text
main:
 /*Enable GPIOC clock, bit 2 on IOPENR*/
 ldr r6, =RCC_IOPENR
 ldr r5, [r6]
 movs r4, 0x4
 orrs r5, r5, r4
 str r5, [r6]
 /* Set up PC6 for the LED (bits 12-13 in MODER)*/
 ldr r6, =GPIOC_MODER
 ldr r5, [r6]
 ldr r4, =0x3000
 mvns r4, r4
 ands r5, r5, r4
 ldr r4, =0x1000
 orrs r5, r5, r4
 str r5, [r6]
 /* Blink the LED*/
 blink_loop:
        ldr r6, =GPIOC_ODR
  /*Read the current state of the LED*/
  ldr r5, [r6]
  /*Set r4 to the value 0x40 (bit 6)*/
```

```
movs r4, 0x40
//Toggle the LED state by XORing with 0x40
eors r5, r5, r4
        // Update the LED state
str r5, [r6]
// Delay between LED state changes (adjust for desired speed)
ldr r0, =DELAY_FREQ  //4_000_000*4 = 16_000_000 cycle it will blink in 1 second
delay_loop:
  subs r0, r0, #1 //This took 1 cycle
  bne delay_loop // This took 3 cycle
// Repeat the blinking loop
b blink_loop
```

3 Problem 2

A button is connected to PA0 pin with pull down mode. When the button is pressed, the input PA0 pin becomes 1 and the LED connected to the PC6 pin on the board lights up. When the button is not pressed, the PA0 pin becomes 0 and the LED connected to the PC6 pin on the board does not light.

3.1 Schematic

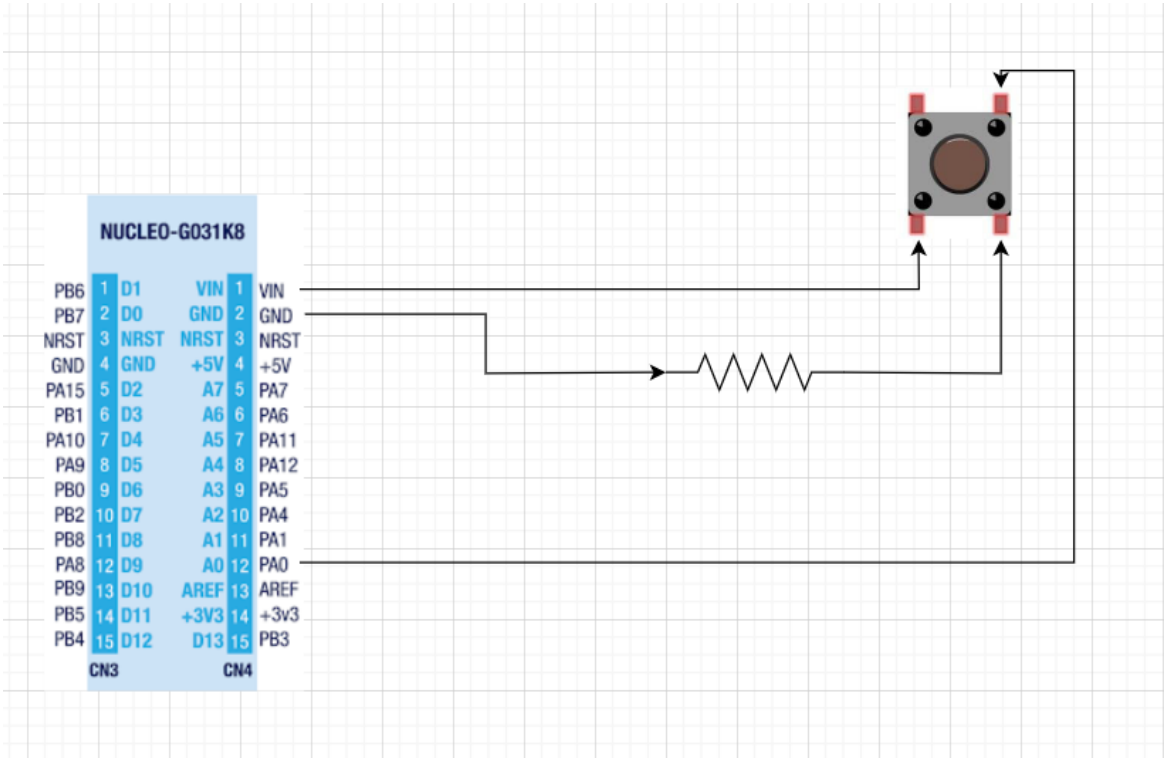


Figure 6: Schematic of problem 2

3.2 Flowchart

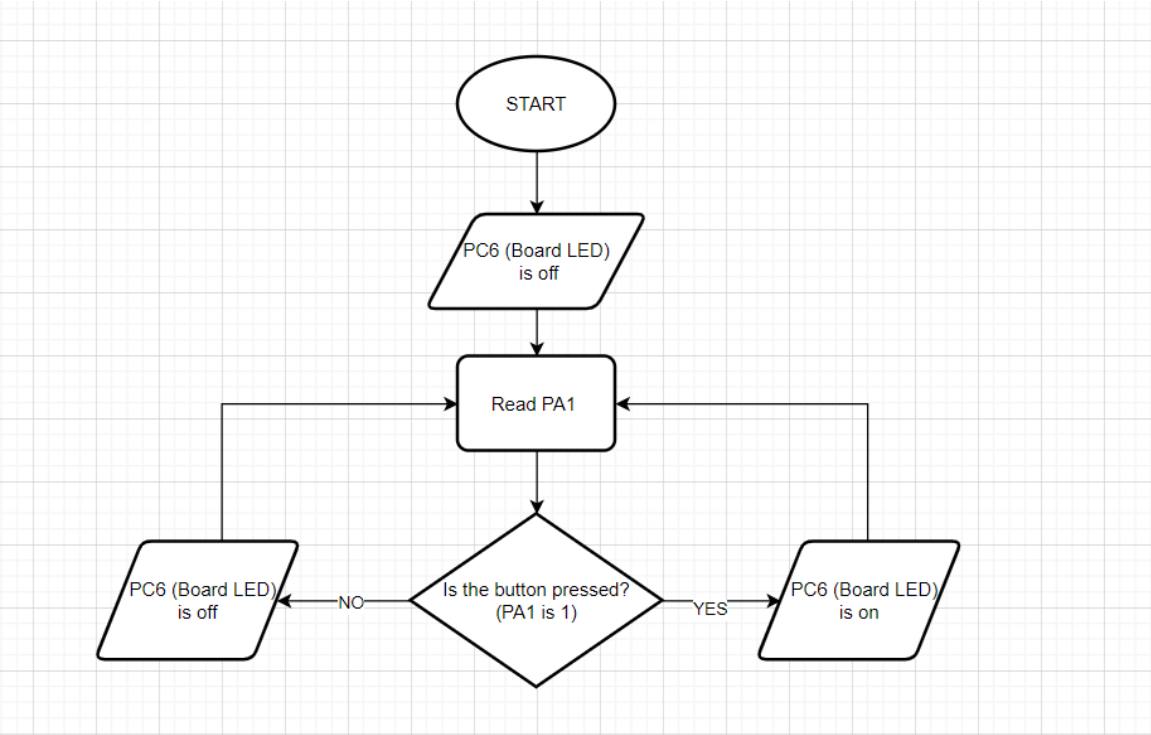


Figure 7: Flowchart of problem 2

3.3 Code

```
/*
* asm.s
*
* author: Fouad Aladhami/Büşra Ülker/Ayşe Beril Cevik
*
* description: Added the necessary stuff for turning on the LED on the
*   G031K8 Nucleo board with button. Mostly for teaching.
*/
.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb
/* make linker see this */
.global Reset_Handler
/* get these from linker script */
.word _sdata
.word _edata
.word _sbss
.word _ebss
/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
.equ RCC_BASE,        (0x40021000)          // RCC base address
.equ RCC_IOPENR,      (RCC_BASE   + (0x34)) // RCC IOPENR register offset
```

```
.equ GPIOA_BASE,      (0x50000000)        // GPIOA base address
.equ GPIOA_MODER,     (GPIOA_BASE + (0x00)) // GPIOA MODER register offset
.equ GPIOA_ODR,       (GPIOA_BASE + (0x14)) // GPIOA ODR register offset
.equ GPIOA_IDR,       (GPIOA_BASE + (0x10)) // GPIOA IDR register offset
.equ GPIOC_BASE,      (0x50000800)        // GPIOC base address
.equ GPIOC_MODER,     (GPIOC_BASE + (0x00)) // GPIOC MODER register offset
.equ GPIOC_ODR,       (GPIOC_BASE + (0x14)) // GPIOC ODR register offset
/* vector table, +1 thumb mode */
.section .vectors
vector_table:
        .word _estack           /*    Stack pointer */
        .word Reset_Handler +1   /*    Reset handler */
        .word Default_Handler +1  /*      NMI handler */
        .word Default_Handler +1  /* HardFault handler */
        /* add rest of them here if needed */
/* reset handler */
.section .text
Reset_Handler:
        /* set stack pointer */
        ldr r0, =_estack
        mov sp, r0
        /* initialize data and bss
         * not necessary for rom only code
         * */
        bl init_data
        /* call main */
        bl main
        /* trap if returned */
        b .
/* initialize data and bss sections */
.section .text
init_data:
        /* copy rom to ram */
        ldr r0, =_sdata
        ldr r1, =_edata
        ldr r2, =_sidata
        movs r3, #0
        b LoopCopyDataInit
        CopyDataInit:
                ldr r4, [r2, r3]
                str r4, [r0, r3]
                adds r3, r3, #4
        LoopCopyDataInit:
                adds r4, r0, r3
                cmp r4, r1
                bcc CopyDataInit
```

```
        /* zero bss */
        ldr r2, =_sbss
        ldr r4, =_ebss
        movs r3, #0
        b LoopFillZerobss
        FillZerobss:
                str  r3, [r2]
                adds r2, r2, #4
        LoopFillZerobss:
                cmp r2, r4
                bcc FillZerobss
        bx lr
/* default handler */
.section .text
Default_Handler:
        b Default_Handler
/* main function */
.section .text
main:
 /*Enable GPIOA/C clock, bit 0 and 2 on IOPENR*/
 ldr r6, =RCC_IOPENR
 ldr r5, [r6]
 movs r4, 0x5//r4=0101
 orrs r5, r5, r4//set 1 bit 0 and 2
 str r5, [r6]
 /* Set up PC6 for the LED (bits 12-13 in MODER)*/
 ldr r6, =GPIOC_MODER
 ldr r5, [r6]
 ldr r4, =0x3000
 bics r5, r5, r4 //erase bits 12 and 13
 ldr r4, =0x1000
 orrs r5, r5, r4 // write 01 to the bits
 str r5, [r6]
 /* Set up PA0 for button the input mode(00)(bits 0-1 in MODER)*/
 ldr r6, =GPIOA_MODER
 ldr r5, [r6]
 ldr r4, =0x3
 bics r5, r5, r4 //erase bits 0 and 1
 str r5, [r6]
 buttoncontrol:
 /*Control button*/
 ldr r6,=GPIOA_IDR//read button
 ldr r5,[r6]
 movs r4,#1 /*r4=0001*/
 ands r5,r5,r4/*if PA0 is 1, r5 is 1*/
 cmp r5,#0x1//Is the button 1?
```

```
beq led_on//Yes.Led on.
bne led_off//No.Led off.
led_on:
ldr r6, =GPIOC_ODR
ldr r5, [r6]
movs r4,#0x40
orrs r5, r5, r4//set pin 6
str r5, [r6]
b buttoncontrol//control button again
led_off:
ldr r6, =GPIOC_ODR
ldr r5, [r6]
movs r4,#0x40
bics r5, r5, r4//clear pin 6
str r5, [r6]
b buttoncontrol//control button again
```

4 Problem 3

The 8 pins of the B port were set to output mode and connected to the LEDs. LEDs were switched on and off with 1 second intervals.

4.1 Schematic

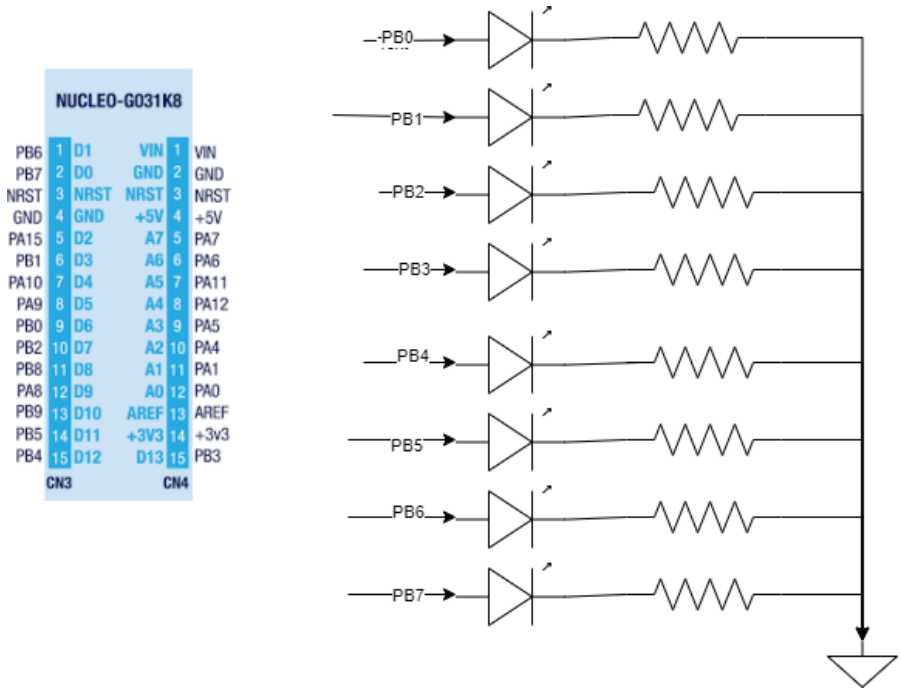


Figure 8: Schematic of problem 3

4.2 Flowchart

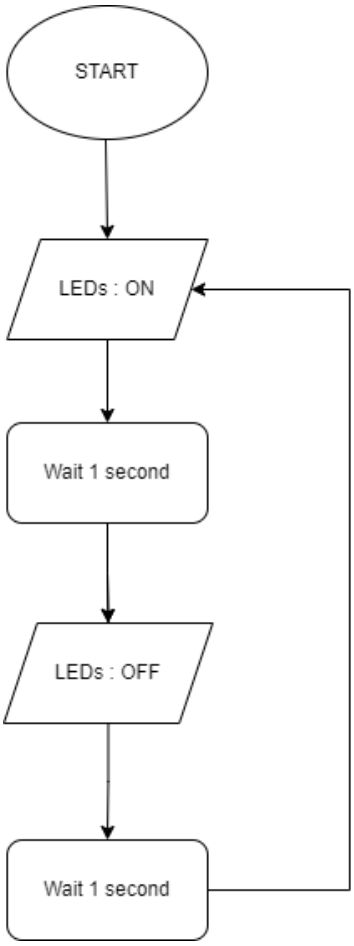


Figure 9: Flowchart of problem 3

4.3 Code

```
/*
* asm.s
*
* author: Fouad Aladhami/Büşra Ülker/Ayşe Beril Cevik
*
* description: Added the necessary stuff for turning on the LED on the
*   G031K8 Nucleo board with button. Mostly for teaching.
*/
.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb
/* make linker see this */
.global Reset_Handler
/* get these from linker script */
.word _sdata
.word _edata
```

```
.word _sbss
.word _ebss
/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
.equ RCC_BASE,        (0x40021000)        // RCC base address
.equ RCC_IOPENR,      (RCC_BASE   + (0x34)) // RCC IOPENR register offset
.equ GPIOA_BASE,      (0x50000000)        // GPIOA base address
.equ GPIOA_MODER,     (GPIOA_BASE + (0x00)) // GPIOA MODER register offset
.equ GPIOA_ODR,       (GPIOA_BASE + (0x14)) // GPIOA ODR register offset
.equ GPIOA_IDR,       (GPIOA_BASE + (0x10)) // GPIOA IDR register offset
.equ GPIOC_BASE,      (0x50000800)        // GPIOC base address
.equ GPIOC_MODER,     (GPIOC_BASE + (0x00)) // GPIOC MODER register offset
.equ GPIOC_ODR,       (GPIOC_BASE + (0x14)) // GPIOC ODR register offset
/* vector table, +1 thumb mode */
.section .vectors
vector_table:
        .word _estack           /*     Stack pointer */
        .word Reset_Handler +1    /*     Reset handler */
        .word Default_Handler +1  /*      NMI handler */
        .word Default_Handler +1  /* HardFault handler */
        /* add rest of them here if needed */
/* reset handler */
.section .text
Reset_Handler:
        /* set stack pointer */
        ldr r0, =_estack
        mov sp, r0
        /* initialize data and bss
         * not necessary for rom only code
         * */
        bl init_data
        /* call main */
        bl main
        /* trap if returned */
        b .
/* initialize data and bss sections */
.section .text
init_data:
        /* copy rom to ram */
        ldr r0, =_sdata
        ldr r1, =_edata
        ldr r2, =_sidata
        movs r3, #0
        b LoopCopyDataInit
        CopyDataInit:
                ldr r4, [r2, r3]
                str r4, [r0, r3]
```

```
                adds r3, r3, #4
        LoopCopyDataInit:
                adds r4, r0, r3
                cmp r4, r1
                bcc CopyDataInit
        /* zero bss */
        ldr r2, =_sbss
        ldr r4, =_ebss
        movs r3, #0
        b LoopFillZerobss
        FillZerobss:
                str  r3, [r2]
                adds r2, r2, #4
        LoopFillZerobss:
                cmp r2, r4
                bcc FillZerobss
        bx lr
/* default handler */
.section .text
Default_Handler:
        b Default_Handler
/* main function */
.section .text
main:
 /*Enable GPIOA/C clock, bit 0 and 2 on IOPENR*/
 ldr r6, =RCC_IOPENR
 ldr r5, [r6]
 movs r4, 0x5//r4=0101
 orrs r5, r5, r4//set 1 bit 0 and 2
 str r5, [r6]
 /* Set up PC6 for the LED (bits 12-13 in MODER)*/
 ldr r6, =GPIOC_MODER
 ldr r5, [r6]
 ldr r4, =0x3000
 bics r5, r5, r4 //erase bits 12 and 13
 ldr r4, =0x1000
 orrs r5, r5, r4 // write 01 to the bits
 str r5, [r6]
 /* Set up PA0 for button the input mode(00)(bits 0-1 in MODER)*/
 ldr r6, =GPIOA_MODER
 ldr r5, [r6]
 ldr r4, =0x3
 bics r5, r5, r4 //erase bits 0 and 1
 str r5, [r6]
 buttoncontrol:
 /*Control button*/
```

```
ldr r6,=GPIOA_IDR//read button
ldr r5,[r6]
movs r4,#1 /*r4=0001*/
ands r5,r5,r4/*if PA0 is 1, r5 is 1*/
cmp r5,#0x1//Is the button 1?
beq led_on//Yes.Led on.
bne led_off//No.Led off.
led_on:
ldr r6, =GPIOC_ODR
ldr r5, [r6]
movs r4,#0x40
orrs r5, r5, r4//set pin 6
str r5, [r6]
b buttoncontrol//control button again
led_off:
ldr r6, =GPIOC_ODR
ldr r5, [r6]
movs r4,#0x40
bics r5, r5, r4//clear pin 6
str r5, [r6]
b buttoncontrol//control button again
```

5 Problem 4

A button is connected to PA8 with pull down mode and the leds is connected to port B starting from PB0 to PB7. First starting from LED0, LED3 was switched on with 100 ms intervals. Than a shift pattern is setted 0000 0111 for 8 bit as initial condition, 1 for leds on and 0 for off. Because registers are 32 bit we will do the pattern as 0000 0111 0000 0111 0000 0111 0000 0111.Than using rotate instruction, the shifting pattern is implemented. When the button pressed the shifting pattern change.

5.1 Schematic

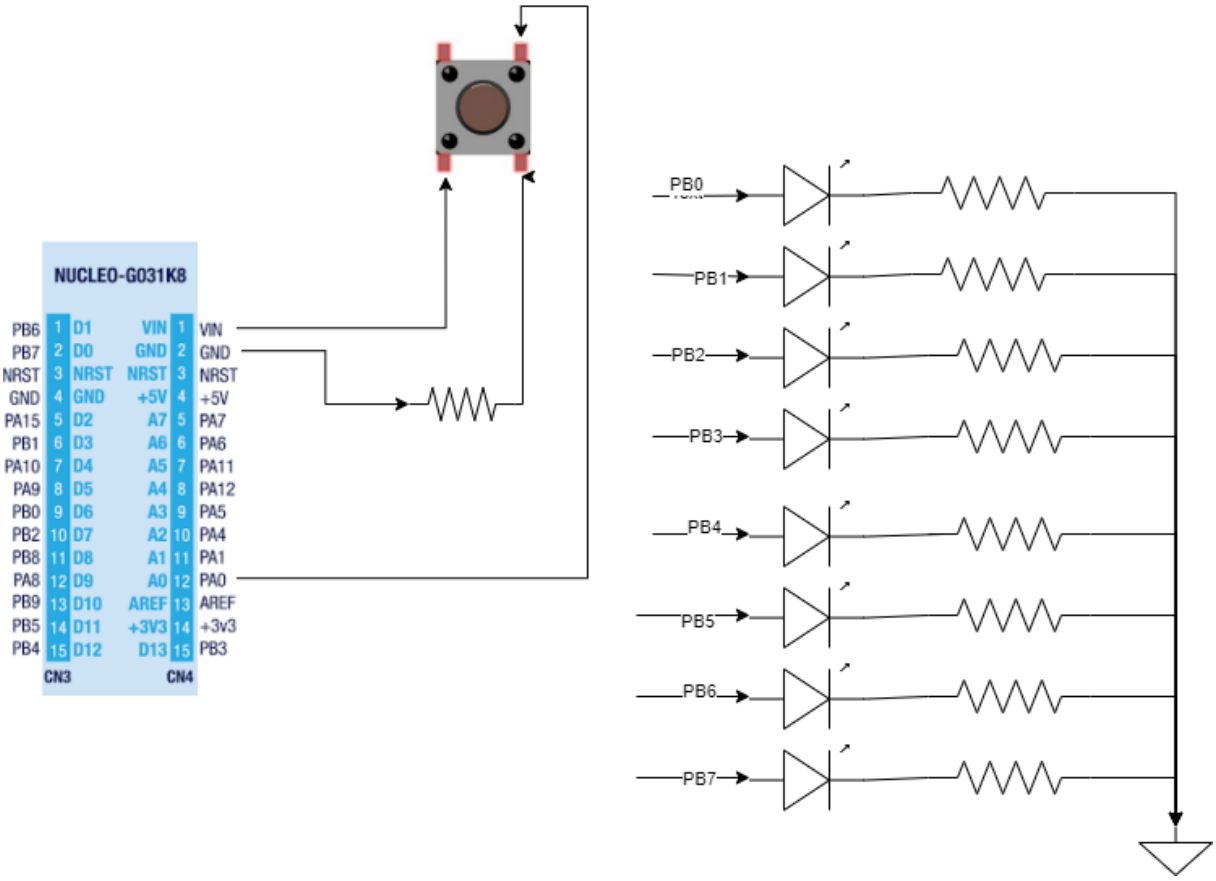


Figure 10: Schematic of problem 4

5.2 Flowchart

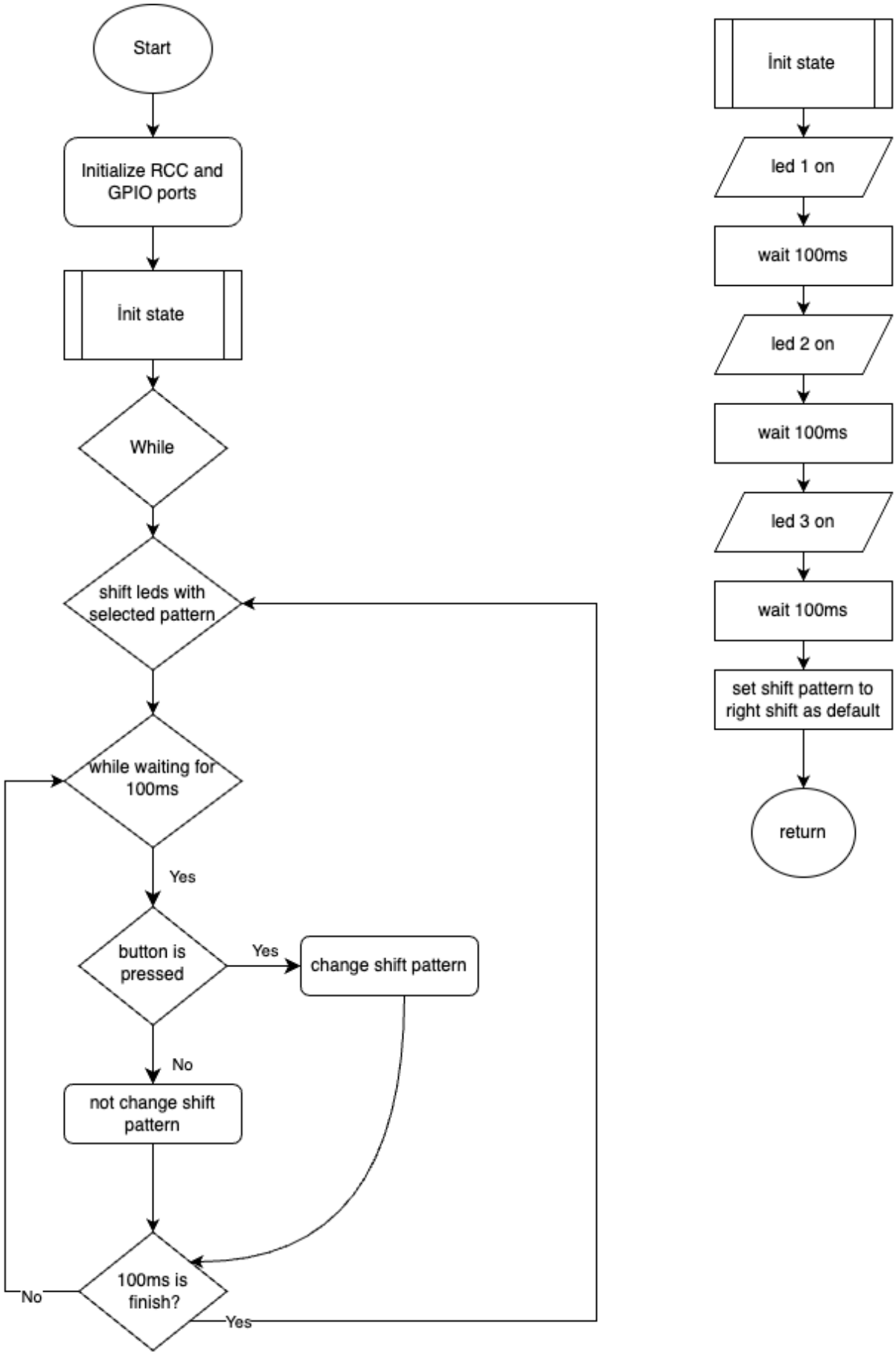


Figure 11: Flowchart of problem 4

5.3 Code

```
/*
* asm.s
*
* author: Fouad Aladhami/Büşra Ülker/Ayşe Beri Çevik
*
* description: Added the necessary stuff for turning on the green LED on the
*   G031K8 Nucleo board. Mostly for teaching.
*/
.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb
/* make linker see this */
.global Reset_Handler
/* get these from linker script */
.word _sdata
.word _edata
.word _sbss
.word _ebss
/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
.equ RCC_BASE,        (0x40021000)        // RCC base address
.equ RCC_IOPENR,      (RCC_BASE   + (0x34)) // RCC IOPENR register offset
.equ GPIOB_BASE,      (0x50000400)        // GPIOB base address
.equ GPIOB_MODER,     (GPIOB_BASE + (0x00)) // GPIOB MODER register offset
.equ GPIOB_ODR,       (GPIOB_BASE + (0x14)) // GPIOB ODR register offset
.equ GPIOA_BASE,      (0x50000000)        // GPIOA base address?
.equ GPIOA_MODER,     (GPIOA_BASE + (0x00)) // GPIOA MODER register offset
.equ GPIOA_ODR,       (GPIOA_BASE + (0x14)) // GPIOA ODR register offset
.equ GPIOA_IDR,       (GPIOA_BASE + (0x10)) // GPIOA IDR register offset
.equ DELAY_FREQ,              (16000000/190)
/* vector table, +1 thumb mode */
.section .vectors
vector_table:
        .word _estack           /*     Stack pointer */
        .word Reset_Handler +1    /*     Reset handler */
        .word Default_Handler +1  /*      NMI handler */
        .word Default_Handler +1  /* HardFault handler */
        /* add rest of them here if needed */
/* reset handler */
.section .text
Reset_Handler:
        /* set stack pointer */
        ldr r0, =_estack
        mov sp, r0
```

```
        /* initialize data and bss
         * not necessary for rom only code
         * */
        bl init_data
        /* call main */
        bl main
        /* trap if returned */
        b .
/* initialize data and bss sections */
.section .text
init_data:
        /* copy rom to ram */
        ldr r0, =_sdata
        ldr r1, =_edata
        ldr r2, =_sidata
        movs r3, #0
        b LoopCopyDataInit
        CopyDataInit:
                ldr r4, [r2, r3]
                str r4, [r0, r3]
                adds r3, r3, #4
        LoopCopyDataInit:
                adds r4, r0, r3
                cmp r4, r1
                bcc CopyDataInit
        /* zero bss */
        ldr r2, =_sbss
        ldr r4, =_ebss
        movs r3, #0
        b LoopFillZerobss
        FillZerobss:
                str  r3, [r2]
                adds r2, r2, #4
        LoopFillZerobss:
                cmp r2, r4
                bcc FillZerobss
        bx lr
/* default handler */
.section .text
Default_Handler:
        b Default_Handler
/* Function to initialize a GPIO pin */
init_gpio:
 // Parameters: r0 = GPIO base address, r1 = pin number, r2 = mode
 ldr r3, [r0]     // Load the current GPIO MODER register value
 lsls r1, r1, #1   //Multiply pin number by 2 for shifting moder mode value
```

```
 lsls r2, r2, r1      // Shift the mode bits to the position of the pin
 movs r4, #0x3 //define r4 mask
 lsls r4, r4, r1 //shift 0x11 mask to the pin register
 bics r3, r3, r4      // Clear the bits for this pin
 orrs r3, r3, r2      // Set the mode bits for this pin
 str r3, [r0]     // Store the updated value back to the MODER register
 bx lr
/* Function to turn on an LED */
turn_on_led:
 // Parameter: r0 = GPIO base address, r1 = pin number
 ldr r2, [r0,#0x14]   // Load the ODR (output data register)
 ldr r3, =#0x01
 lsls r3, r3, r1      // Create a bitmask for the pin
 orrs r2, r2, r3      // Set the pin to 1 (turn on the LED)
 str r2, [r0,#0x14]   // Store the updated value back to the ODR register
 bx lr
/* Function to turn off an LED */
turn_off_led:
 // Parameter: r0 = GPIO base address, r1 = pin number
 ldr r2, [r0, #0x14]   // Load the ODR (output data register)
 ldr r3, =#0x01
 lsls r3, r3, r1      // Create a bitmask for the pin
 bics r2, r2, r3      // Clear the pin to 0 (turn off the LED)
 str r2, [r0, #0x14]   // Store the updated value back to the ODR register
 bx lr
rcc_gpio_clock_enable:
        // Parameters: r0 = RCC_IOPENR base address, r1 = GPIO port
        ldr r3, [r0] //Load the RCC_IOPENR value
        //make a mask for the port
        movs r4, #0x1
        lsls r4, r4, r1
        //set the port bit
        orrs r3, r3, r4
        //store the updated value back to the RCC_IOPENR register
        str r3, [r0]
        bx lr
delay_loop:
        //load counter to r0 register
        button_control:
                /*Control button*/
                ldr r2,=GPIOA_IDR //2 cycle
                ldr r1,[r2]        //2 cycle
                movs r3,#1 /*r4=0001*/ //1cycle
                ands r1,r1,r3/*if PA0 is 1, r5 is 1*/ //1cycle
                movs r7,r1 //1cycle
                movs r3,0 //1cycle
```

```
                mvns r3,r6 //1cycle
                ands r3, r7 //1cycle
                bne change_pattern //1cycle if condition not-taken
                beq do_nothing//3 cycle if condition tken
                do_nothing:
                        movs r6,r7      //this took 1 cycle
                        subs r0, r0, #1 //This took 1 cycle
                        bne delay_loop // This took 3 cycle
                        bx lr
                change_pattern:
                        movs r6,r7
                        cmp r4,#1
                        beq set_shift_right_pattern
                        bne set_shift_left_pattern
                        set_shift_right_pattern:
                                movs r4, #31
                                b pattern_changed
                        set_shift_left_pattern:
                                movs r4,#1
                        pattern_changed:
                                subs r0, r0, #1 //This took 1 cycle
                                bne delay_loop // This took 3 cycle
                                bx lr
/* main function */
.section .text
main:
        /*Enable GPIOA clock, bit 0 on IOPENR*/
        ldr r0, =RCC_IOPENR
        movs r1, #0 //set r1 to 1 for GPIOA
        bl rcc_gpio_clock_enable //call the rcc_gpio_clock_enable function
        /*Enable GPIOB clock, bit 1 on IOPENR*/
        ldr r0, =RCC_IOPENR
        movs r1, #1 //set r1 to 1 for GPIOB
        bl rcc_gpio_clock_enable //call the rcc_gpio_clock_enable function
        /* Set up PB0- PB7 for the LED (bits 16-17 in MODER)*/
        ldr r6, =GPIOB_MODER
        ldr r5, [r6]
        ldr r4, =0xFFFF
        bics r5, r5, r4 //erase bits 16 and 17
        ldr r4, =0x5555
        orrs r5, r5, r4 // write 01 to the bits
        str r5, [r6]
        ldr r0, =GPIOA_MODER
        movs r1, #0 //0 for active pin 0
        movs r2, #0 //0 for input mode
        bl init_gpio
```

```
movs r6,#0
movs r7,#0
initstate:
        ldr r0, =GPIOB_BASE
        movs r1, #0 //0 for pin 0
        bl turn_on_led
        ldr r0, =DELAY_FREQ
        bl delay_loop
        ldr r0, =GPIOB_BASE
        movs r1, #1 //1 for pin 1
        bl turn_on_led
        ldr r0, =DELAY_FREQ
        bl delay_loop
        ldr r0, =GPIOB_BASE
        movs r1, #2 //2 for pin 2
        bl turn_on_led
        ldr r0, =DELAY_FREQ
        bl delay_loop
        ldr r5,=0x07070707 // assign led pattern to r5
        movs r4, #31
while_loop:
        rors r5,r5,r4
        ldr r1, =GPIOB_ODR
        str r5, [r1]
        ldr r0, =DELAY_FREQ
        bl delay_loop
        bl while_loop
```

6 Problem 5

Leds was connected to the pins of port B. Firstly, starting from LED0, LED3 was switched on with 100ms intervals. Then the led numbers were shifted to the right with 100ms intervals. With the algorithm created, when LED8 was reached, it was shifted towards LED0, and when LED0 was reached, it was shifted towards LED8.

6.1 Schematic

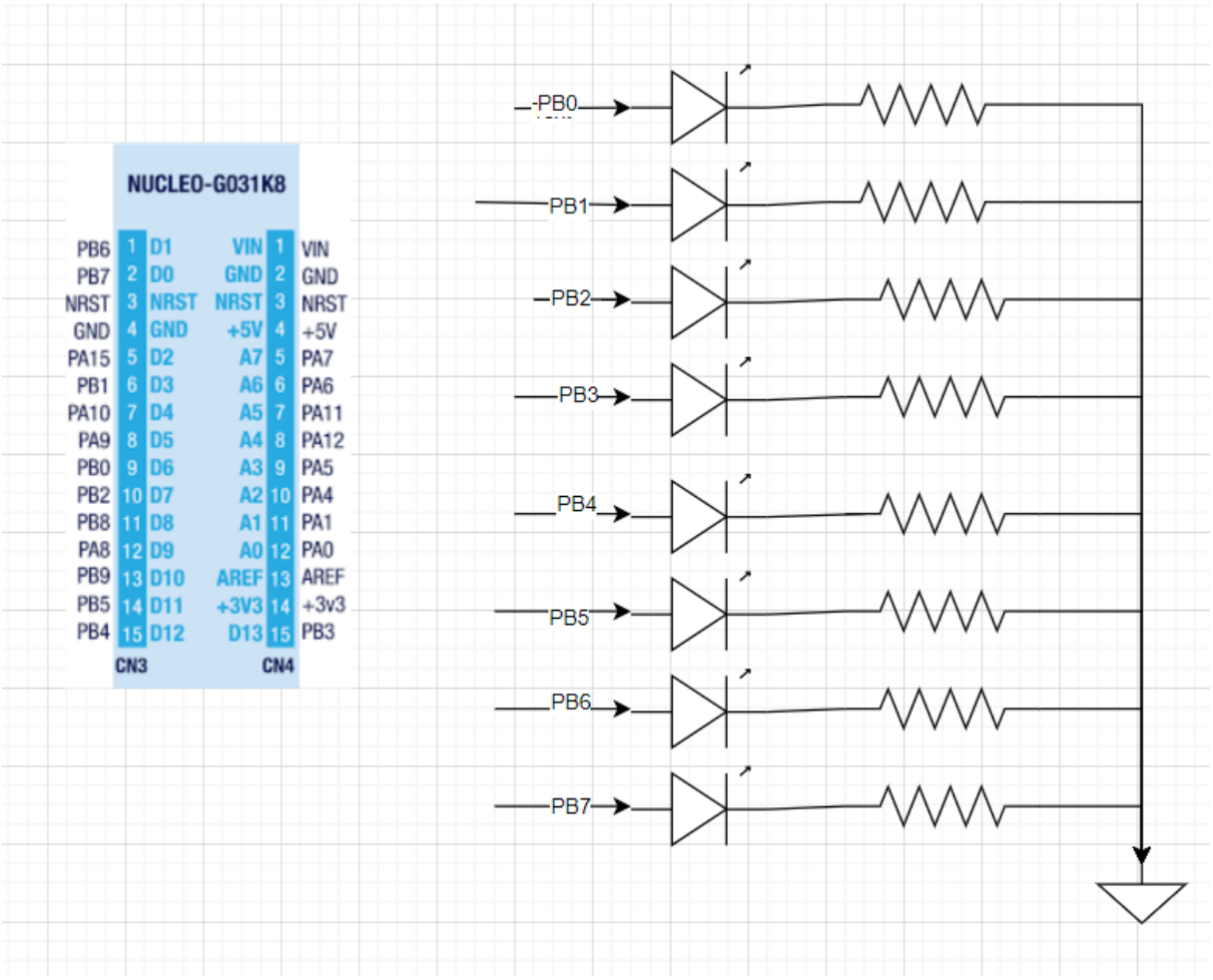


Figure 12: Schematic of problem 5

6.2 Flowchart

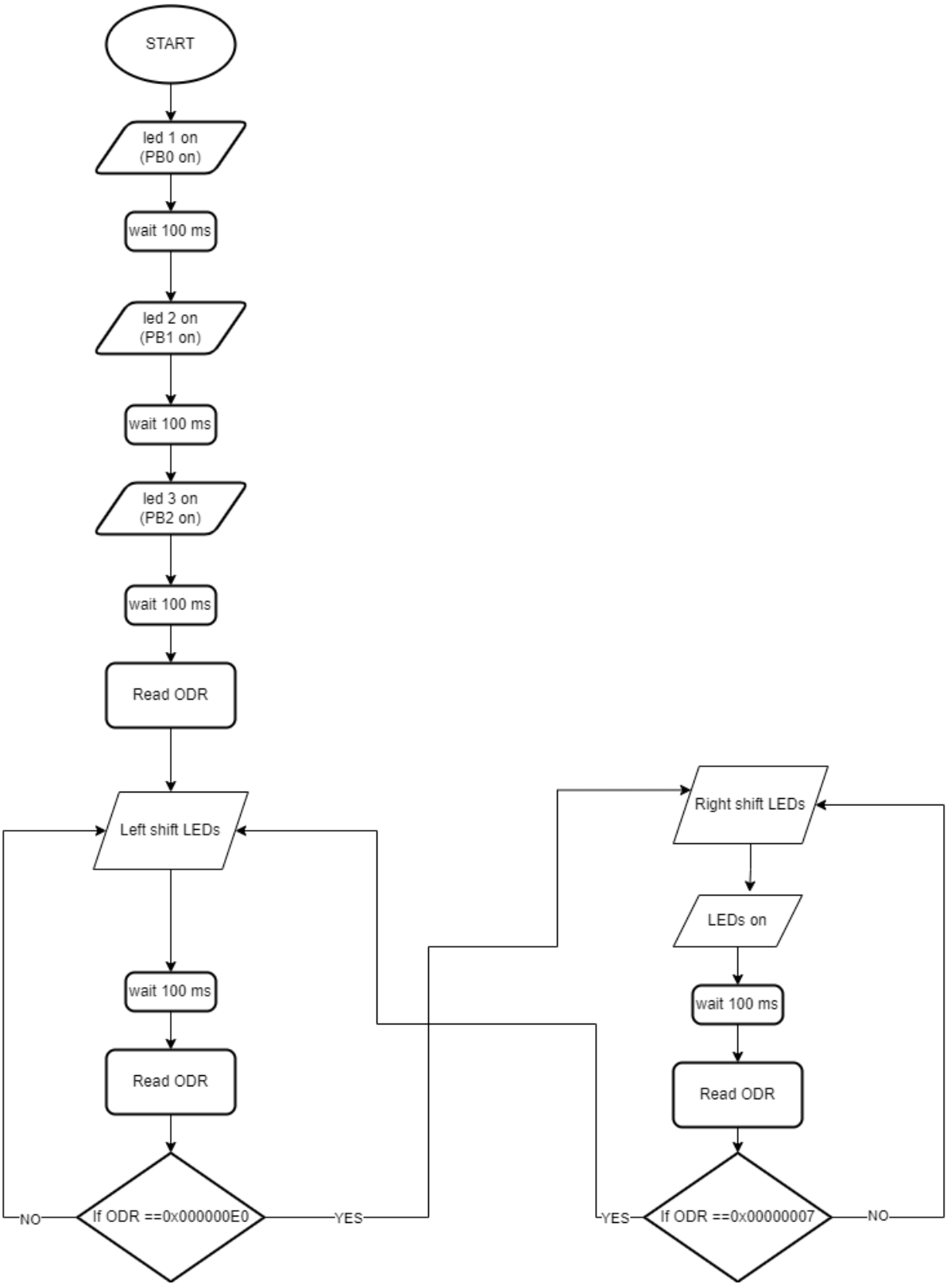


Figure 13: Flowchart of problem 5

6.3 Code

```
/*
* asm.s
*
* author: Fouad Aladhami/Büşra Ülker/Ayşe Beri Çevik
*
* description: Added the necessary stuff for turning on  LEDS with
*   G031K8 Nucleo board.
*/
.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb
/* make linker see this */
.global Reset_Handler
/* get these from linker script */
.word _sdata
.word _edata
.word _sbss
.word _ebss
/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
.equ RCC_BASE,        (0x40021000)        // RCC base address
.equ RCC_IOPENR,      (RCC_BASE   + (0x34)) // RCC IOPENR register offset
.equ GPIOB_BASE,      (0x50000400)        // GPIOB base address
.equ GPIOB_MODER,     (GPIOB_BASE + (0x00)) // GPIOB MODER register offset
.equ GPIOB_ODR,       (GPIOB_BASE + (0x14)) // GPIOB ODR register offset
.equ DELAY_FREQ,              (16000000/30)
/* vector table, +1 thumb mode */
.section .vectors
vector_table:
        .word _estack           /*     Stack pointer */
        .word Reset_Handler +1    /*     Reset handler */
        .word Default_Handler +1  /*      NMI handler */
        .word Default_Handler +1  /* HardFault handler */
        /* add rest of them here if needed */
/* reset handler */
.section .text
Reset_Handler:
        /* set stack pointer */
        ldr r0, =_estack
        mov sp, r0
        /* initialize data and bss
         * not necessary for rom only code
         * */
        bl init_data
```

```
        /* call main */
        bl main
        /* trap if returned */
        b .
/* initialize data and bss sections */
.section .text
init_data:
        /* copy rom to ram */
        ldr r0, =_sdata
        ldr r1, =_edata
        ldr r2, =_sidata
        movs r3, #0
        b LoopCopyDataInit
        CopyDataInit:
                ldr r4, [r2, r3]
                str r4, [r0, r3]
                adds r3, r3, #4
        LoopCopyDataInit:
                adds r4, r0, r3
                cmp r4, r1
                bcc CopyDataInit
        /* zero bss */
        ldr r2, =_sbss
        ldr r4, =_ebss
        movs r3, #0
        b LoopFillZerobss
        FillZerobss:
                str  r3, [r2]
                adds r2, r2, #4
        LoopFillZerobss:
                cmp r2, r4
                bcc FillZerobss
        bx lr
/* default handler */
.section .text
Default_Handler:
        b Default_Handler
/* Function to initialize a GPIO pin */
init_gpio:
 // Parameters: r0 = GPIO base address, r1 = pin number, r2 = mode
 ldr r3, [r0]     // Load the current GPIO MODER register value
 lsls r1, r1, #1   //Multiply pin number by 2 for shifting moder mode value
 lsls r2, r2, r1      // Shift the mode bits to the position of the pin
 movs r4, #0x3 //define r4 mask
 lsls r4, r4, r1 //shift 0x11 mask to the pin register
 bics r3, r3, r4      // Clear the bits for this pin
```

```
 orrs r3, r3, r2      // Set the mode bits for this pin
 str r3, [r0]     // Store the updated value back to the MODER register
 bx lr
/* Function to turn on an LED */
turn_on_led:
 // Parameter: r0 = GPIO base address, r1 = pin number
 ldr r2, [r0,#0x14]   // Load the ODR (output data register)
 ldr r3, =#0x01
 lsls r3, r3, r1      // Create a bitmask for the pin
 orrs r2, r2, r3      // Set the pin to 1 (turn on the LED)
 str r2, [r0,#0x14]   // Store the updated value back to the ODR register
 bx lr
/* Function to turn off an LED */
turn_off_led:
 // Parameter: r0 = GPIO base address, r1 = pin number
 ldr r2, [r0, #0x14]   // Load the ODR (output data register)
 ldr r3, =#0x01
 lsls r3, r3, r1     // Create a bitmask for the pin
 bics r2, r2, r3      // Clear the pin to 0 (turn off the LED)
 str r2, [r0, #0x14]   // Store the updated value back to the ODR register
 bx lr
rcc_gpio_clock_enable:
          // Parameters: r0 = RCC_IOPENR base address, r1 = GPIO port
          ldr r3, [r0] //Load the RCC_IOPENR value
          //make a mask for the port
          movs r4, #0x1
          lsls r4, r4, r1
          //set the port bit
          orrs r3, r3, r4
          //store the updated value back to the RCC_IOPENR register
          str r3, [r0]
          bx lr
delay_loop:
          //load counter to r0 register
          subs r0, r0, #1 //This took 1 cycle
  bne delay_loop // This took 3 cycle
  bx lr
delay_loop1:
          //load counter to r0 register
          subs r0, r0, #1 //This took 1 cycle
  bne delay_loop1 // This took 3 cycle
  b left_shift
delay_loop2:
          //load counter to r0 register
          subs r0, r0, #1 //This took 1 cycle
  bne delay_loop2 // This took 3 cycle
```

```
  b right_shift
/* main function */
.section .text
main:
        /*Enable GPIOA clock, bit 0 on IOPENR*/
        ldr r0, =RCC_IOPENR
        movs r1, #0 //set r1 to 1 for GPIOA
        bl rcc_gpio_clock_enable //call the rcc_gpio_clock_enable function
        /*Enable GPIOB clock, bit 1 on IOPENR*/
        ldr r0, =RCC_IOPENR
        movs r1, #1 //set r1 to 1 for GPIOB
        bl rcc_gpio_clock_enable //call the rcc_gpio_clock_enable function
        /* Set up PB0- PB7 for the LED (bits 16-17 in MODER)*/
        ldr r6, =GPIOB_MODER
        ldr r5, [r6]
        ldr r4, =0xFFFF
        bics r5, r5, r4 //erase bits 16 and 17
        ldr r4, =0x5555
        orrs r5, r5, r4 // write 01 to the bits
        str r5, [r6]
        initstate:
        /*İnitial codes for the first 3 LEDs of the system to light up*/
                ldr r0, =GPIOB_BASE
                movs r1, #0 //0 for pin 0
                bl turn_on_led
                ldr r0, =DELAY_FREQ//for 100 ms delay
                bl delay_loop
                ldr r0, =GPIOB_BASE
                movs r1, #1 //1 for pin 1
                bl turn_on_led
                ldr r0, =DELAY_FREQ//for 100 ms delay
                bl delay_loop
                ldr r0, =GPIOB_BASE
                movs r1, #2 //2 for pin 2
                bl turn_on_led
                ldr r0, =DELAY_FREQ//for 100 ms delay
                bl delay_loop
                ldr r5,=0x00000007 // assign led pattern to r5
                ldr r4,=0x1
                ldr r6,=0x000000E0 //left limit
                ldr r7,=0x00000007 //right limit
        left_shift:
        cmp r5,r6//It is checked whether it reaches the leftmost bit or not.
        beq right_shift//Shift right if on the leftmost bit
        lsls r5,r5,r4//shift left 1 bit
        ldr r1, =GPIOB_ODR
```

```
str r5, [r1]//Write ODR
ldr r0, =DELAY_FREQ//for 100ms delay
b delay_loop1
right_shift:
cmp r5,r7//It is checked whether it reaches the rightmost bit or not.
beq left_shift//Shift left if on the rightmost bit
lsrs r5,r5,r4//shift right 1 bit
ldr r1, =GPIOB_ODR
str r5, [r1]//Write ODR
ldr r0, =DELAY_FREQ//for 100 ms delay
b delay_loop2
```